Event Types

This document provides information about Misty's available event types. Use these event types to get live updates with data from Misty's sensors and other system services.

To receive event messages, you can register event listeners in your skill code. You can also subscribe to event messages from a remote device by connecting to Misty's WebSocket server.

When you register for event messages, you can:

- filter out unwanted data by specifying which property values an event message should include
- apply conditions to receive messages only when the property values for an event meet specific criteria

Note: If your Misty is using the `Current` version of Misty's WebSocket system, WebSocket event messages do not include `SensorName` or `Type` key/value pairs. Use Misty's GetWebsocketVersion command to find out which version your robot is using, and use SetWebsocketVersion to switch versions.

Tip: Some event types (such as `TimeOfFlight` and `ActuatorPosition`) stream event messages from a group of related sensors. When you set up a generalized event listener for these event types, you receive messages from every sensor in the group. Because the system defaults to reporting the message from the sensor with the most recently updated data, this can result in a situation where one sensor appears to send messages more frequently than the other sensors in that group, even when using very low debounce values.

To avoid this, we recommend using **property tests**/**event conditions** when you register event listeners for event types that report data from more than one sensor. This allows you to configure a listener to handle messages only from one specific sensor in a group. You can register multiple event listeners - each with unique property tests - for any single event type. In this manner, you can create a group of event listeners for a single event type, where each listener is assigned to trigger an event callback only on receiving messages from one specific sensor associated with that event type.

ActuatorPosition

The `ActuatorPosition` event type provides information about the position of the actuators responsible for controlling the movement of Misty's head and arms. `ActuatorPosition` data is sent at timed intervals you define when you register for `ActuatorPosition` messages.

In the `ActuatorPosition` data object, the value of the `sensorName` property is the name of the actuator you are receiving information about (`Actuator_HeadPitch`, `Actuator_HeadYaw`, `Actuator_HeadRoll`, `Actuator_LeftArm`, or `Actuator_RightArm`). The `value` property holds a number indicating the position of the actuator (in degrees).

When you subscribe to the `ActuatorPosition` data stream, you should specify which actuator you want to receive messages about. For example, the following sample code shows how to use a property comparison test to get data from the sensor for the actuator

responsible for controlling the movement of Misty's right arm with the on-robot JavaScript API:

```
// Register for ActuatorPosition data for the actuator for Misty's right arm
misty.AddPropertyTest("ActuatorPosition", "sensorName", "==",
"Actuator_RightArm", "string");
misty.RegisterEvent("ActuatorPosition", "ActuatorPosition", 1000, true);

// Callback function that triggers each time ActuatorPosition data is sent
function _ActuatorPosition(data) {
// Do something with the data
}
```

JavaScript

Sample `ActuatorPosition` data:

```
ActuatorPosition {
     "eventName": "ActuatorPosition",
     "message": {
          "actuatorId": "NBQEA0",
          "created": "2019-01-09T20:15:38.0870356Z",
          "expiry": "2019-01-09T20:15:39.0870356Z",
          "sensorId": "ala",
          "sensorName": "Actuator_LeftArm",
          "value":-5.09
     },
     "type":"ActuatorPosition"
 }
```

JSON

AudioPlayComplete

`AudioPlayComplete` WebSocket data is sent every time Misty finishes playing an audio file. It is not sent at timed intervals.

```
AudioPlayComplete {
     "eventName":"AudioPlayComplete",
     "message":{
          "created":"2019-04-08T20:54:36.7051135Z",
          "metaData":{
               "directory":"Idle",
               "duration":0,
               "name":"002-Ahhh.wav",
               "vad":[0,0,-0.5]
          }
     }
 }
```

JSON

BatteryCharge

The `BatteryCharge` event type provides a stream of information about the current state of Misty's battery, including charge percentage, voltage, and charging status. The `BatteryCharge` event message includes the following key/value pairs:

- `chargePercent` (double) - Decimal value representing current charge percent.
- `created` (string) - Timestamp that describes when the system created this message.
- `current` (int) - The current flowing into or discharging from the battery. This value is negative when the battery is discharging, and positive when the battery is being charged.

- `expiry` (string) - Timestamp describing the moment after which the values in this message should no longer be considered valid.
- `healthPercent` (double)
- `isCharging` (bool) - Returns `true` if the battery is charging. Otherwise, `false`.
- `sensorId` (string) - The `sensorId` of the system component that returns the battery charge message (`charge`).
- `state` (string) - The charge state of the battery. Possible values are:
  - `Charging` (if battery is receiving current)
  - `Discharging` (if battery is losing current)
  - `Charged` (if battery is fully charged)
  - `Unknown` (if you check the charge levels before Misty is fully booted, or if the RT board resets and the system has not yet learned the actual battery state)
  - `Fault` (can occur if the charger does not detect the battery)
- `temperature` (int)
- `trained` (bool) - Returns `true` if the battery has been trained. Otherwise, `false`.
- `voltage` (double) - The battery's voltage.

Sample `BatteryCharge` data:


```
{
  "eventName": "BatteryChargeEvent",
  "message": {
    "chargePercent": null,
    "created": "2019-07-23T16:49:27.558817Z",
    "current": -0.441,
    "healthPercent": null,
    "isCharging": false,
    "sensorId": "charge",
    "state": "Discharging",
    "temperature": 83,
    "trained": false,
    "voltage": 8.203
  }
}
```


JSON

BumpSensor

The `BumpSensor` data stream sends information each time one of the bump sensors on Misty's base is pressed or released. In the `BumpSensor` data object, the value of the `sensorName` property is the name of the bump sensor that triggered the event (`Bump_FrontRight`, `Bump_FrontLeft`, `Bump_RearRight`, or `Bump_RearLeft`). The value of the `isContacted` property is a boolean indicating whether the bump sensor was pressed (`true`) or released (`false`).The `BumpSensor` data object also provides “pose” information about Misty at the time of the event. For more about pose, see the mapping section of the Command Center documentation.

For an example that shows how to register for and use data from `BumpSensor` events with Misty's on-robot JavaScript API, see the Bump Sensors skill tutorial.

Sample `BumpSensor` data:


```
BumpSensor {
    "eventName": "BumpSensor",
    "message": {
        "created": "2019-01-09T20:32:27.1036178Z",
        "expiry": "2019-01-09T20:32:27.2286178Z",
        "isContacted": true,
        "pose": {
            "bearing": -0.5585993432483588,
```

```
"created": "2019-01-09T16:36:46.7089139Z",
"distance": 0.14150971628586034,
"elevation": 0,
"frameId": "RobotBaseCenter",
"framesProvider": {
    "rootFrame": {
        "created": "2019-01-09T16:36:46.6151597Z",
        "id": "RobotBaseCenter",
        "isStatic": true,
        "linkFromParent": {
            "isStatic": true,
            "parentFrameId": "",
            "transformFromParent": {
                "bearing": 0,
                "distance": 0,
                "elevation": 0,
                "pitch": 0,
                "quaternion": {
                    "isIdentity": true,
                    "w": 1,
                    "x": 0,
                    "y": 0,
                    "z": 0
                },
                "roll": 0,
                "x": 0,
                "y": 0,
                "yaw": 0,
                "z": 0
            },
            "transformToParent": {
                "bearing": 3.141592653589793,
                "distance": 0,
                "elevation": 0,
                "pitch": 0,
                "quaternion": {
                    "isIdentity": true,
                    "w": 1,
                    "x": 0,
                    "y": 0,
                    "z": 0
                },
                "roll": 0,
                "x": 0,
                "y": 0,
                "yaw": 0,
                "z": 0
            }
        }
    },
    "homogeneousCoordinates": {
        "bearing": -0.5585993432483588,
        "distance": 0.14150971628586034,
        "elevation": 0,
        "pitch": 0,
        "quaternion": {
            "isIdentity": true,
            "w": 1,
            "x": 0,
            "y": 0,
            "z": 0
        },
        "roll": 0,
        "x": 0.11999999731779099,
        "y": -0.07500000298023224,
        "yaw": 0,
        "z": 0
```

```
                },
                "pitch": 0,
                "roll": 0,
                "x": 0.11999999731779099,
                "y": -0.07500000298023224,
                "yaw": 0,
                "z": 0
            },
            "sensorId": "bfr",
            "sensorName": "Bump_FrontRight"
        },
        "type":"BumpSensor"
 }
```

JSON

DriveEncoders

The `DriveEncoders` data stream provides information about the angular velocity (in degrees per second) and rotation (in degrees) for Misty's left and right encoders. `DriveEncoders` data is sent at timed intervals you define when you register for `DriveEncoders` messages.

Sample `DriveEncoders` sensor data:

```
{
   "eventName": "DriveEncodersEvent",
   "message": {
      "created": "2019-07-23T16:54:05.7372361Z",
      "leftDistance": 626,
      "leftVelocity": 249,
      "rightDistance": 676,
      "rightVelocity": 247,
      "sensorId": "enc"
   }
}
```

JSON

FaceRecognition

You can subscribe to the `FaceRecognition` WebSocket to obtain data on both face detection and face recognition events.

The `EventName` value is the name you provide when you register the WebSocket connection.

If face recognition is running on the robot, and a previously trained face is recognized, the `PersonName` value is the name previously assigned to that face. The `PersonName` value is `unknown_person` if an untrained/unknown face is detected. The `PersonName` value is `null` if face recognition is not currently running.

`TrackId` is reserved data that may change in the future.

Sample FaceRecognition data for a face recognition event:

```
FaceRecognition{
     "EventName":"MyFaceRecognition",
     "Message":{
          "Bearing":-3,
          "Created":"2018-07-02T16:26:20.1718422Z",
          "Distance":71,
          "Elevation":3,
          "Expiry":"2018-07-02T16:26:20.9218446Z",
          "PersonName":"Barkley",
          "SensorId":null,
          "SensorName":null,
```

```
            "TrackId":0
        },
        "Type":"FaceRecognition"
    }
```

JavaScript

FaceTraining

The `FaceTraining` event type sends messages from Misty's computer vision service with information about the status the face training process.

> **Tip:** To start the face training process, you must issue a `StartFaceTraining` command.

In addition to `created` and `sensorId` fields, `FaceTraining` messages include the following properties:

- `isProcessComplete` (boolean) - Returns `true` when the face training process is complete or cancelled due to an error or timing out.
- `message` (string) - A message from Misty's computer vision service with information about the status of the face training process. Includes `Error` messages (i.e. when the face training process times out, or when it can't be started because face training is already underway or the value passed in for the `FaceId` argument is already in use), `Warning` messages (i.e. if Misty detects no faces, more than one face, or if the detected face is too far away), and `Status` messages (i.e. which phase of training is currently underway).
- `messageType` (string) - A type label for the sent message (i.e. `Warning`, `Error`, or `Status`).

```
// Sample FaceTraining event message
{
  "eventName":"FaceTraining",
  "message": {
    "created":"2019-09-17T19:31:39.2670814Z",
    "isProcessComplete":false,
    "message":"Face training detection phase complete.",
    "messageType":"Status",
    "sensorId":"cv"
  }
}
```

JSON

HaltCommand

`HaltCommand` WebSocket data is sent every time the robot stops and contains the date and time of the event. It is not sent at timed intervals.

HazardNotification

Misty sends `HazardNotification` event messages each time the hazard system detects a change to a hazard state. You can subscribe to `HazardNotification` events to be notified when Misty enters or exits a specific hazard state. Use this data to have Misty programmatically alter her course or change her behavior in skills that use automated movement and locomotion.

> **Note:** This event type is currently in **Alpha**, and related hardware, firmware, or software is still under development. Feel free to use this event type, but recognize that it may behave unpredictably at this time.

Each `HazardNotification` event message includes the following properties:

- `bumpSensorsHazardState` - An array of objects that describe the hazard state for each of Misty's bump sensors. Each object in the array includes a `sensorName` property with a string value for a bump sensor name, and an `inHazard` property with a boolean value indicating whether that bump sensor is in a hazard state. The hazard system considers a bump sensor to be in a hazard state when that sensor is activated/pressed.
- `criticalInternalError` - A number indicating the presence of a critical internal error that could prevent the hazards system from functioning. If 0, there is no critical internal error; if 1, there is a communication issue between the firmware and the 410 processor in Misty's headboard; if 2, the system has lost communications with Misty's real-time controller. If you encounter a `criticalInternalError` with a value of 1 or 2, please contact a member of the Misty support team.
- `currentSensorsHazard` - The contents of this property are still under implementation and, for now, can be safely ignored.
- `driveStopped` - An array of objects that describe the hazard state for six zones around Misty. Each object in the array includes a `sensorName` property with a string value for a zone, and an `inHazard` property with a boolean value indicating whether that zone is in a hazard state. Data from Misty's time-of-flight and bump sensors can trigger a `driveStopped` hazard state.
- `motorStallHazard` - The contents of this property are still under implementation and, for now, can be safely ignored.
- `timeOfFlightSensorsHazardState` - An array of objects that describe the hazard state for each of Misty's time-of-flight sensors. Each object in the array includes a `sensorName` property with a string value for a time-of-flight sensor name, and an `inHazard` property with a boolean value indicating whether that sensor is in a hazard state. The hazard system considers a time-of-flight sensor to be in a hazard state when the sensor detects a distance greater than a predetermined threshold. For downward-facing time-of-flight sensors, this threshold defaults to 0.06 meters. The hazard system is disabled for front- and rear-facing time-of-flight sensors at this time.
- `excessiveSpeedHazard` - An array of objects that describe whether the velocity for a motor has exceeded safe thresholds, thus triggering a hazard state for that motor. The contents of this property are still under implementation, and the `excessiveSpeedHazard` state is not yet enabled for Misty's head and arm motors. Currently, the system only engages this hazard state when it detects the robot turning at speeds greater than 100 degrees per second. This can sometimes happen as the result of the drive system losing communication with the IMU sensor.

In the following example of a `HazardNotification` event message, the `TOF_DownBackLeft` and `TOF_DownBackRight` time-of-flight sensors are both in a hazard state.


```
{
  "eventName": "HazardNotification",
  "message": {
    "bumpSensorsHazardState": [
      {
        "inHazard": false,
        "sensorName": "Bump_FrontRight"
      },
      {
        "inHazard": false,
        "sensorName": "Bump_FrontLeft"
      },
      {
        "inHazard": false,
        "sensorName": "Bump_RearRight"
      },
      {
```

```
      "inHazard": false,
      "sensorName": "Bump_RearLeft"
    }
  ],
  "criticalInternalError": 0,
  "currentSensorsHazard": [
    {
      "inHazard": false,
      "sensorName": "CurrentSensor_RightTrack"
    },
    {
      "inHazard": false,
      "sensorName": "CurrentSensor_LeftTrack"
    },
    {
      "inHazard": false,
      "sensorName": "CurrentSensor_RightArm"
    },
    {
      "inHazard": false,
      "sensorName": "CurrentSensor_LeftArm"
    },
    {
      "inHazard": false,
      "sensorName": "CurrentSensor_HeadPitch"
    },
    {
      "inHazard": false,
      "sensorName": "CurrentSensor_HeadRoll"
    },
    {
      "inHazard": false,
      "sensorName": "CurrentSensor_HeadYaw"
    }
  ],
  "driveStopped": [
    {
      "inHazard": false,
      "sensorName": "Front right hazard"
    },
    {
      "inHazard": false,
      "sensorName": "Front center hazard"
    },
    {
      "inHazard": false,
      "sensorName": "Front left hazard"
    },
    {
      "inHazard": false,
      "sensorName": "Back right hazard"
    },
    {
      "inHazard": false,
      "sensorName": "Back center hazard"
    },
    {
      "inHazard": false,
      "sensorName": "Back left hazard"
    }
  ],
  "motorStallHazard": [
    {
      "inHazard": false,
      "sensorName": "RightDrive"
    },
    {
      "inHazard": false,
```

```
        "sensorName": "LeftDrive"
      },
      {
        "inHazard": false,
        "sensorName": "RightArm"
      },
      {
        "inHazard": false,
        "sensorName": "LeftArm"
      },
      {
        "inHazard": false,
        "sensorName": "HeadPitch"
      },
      {
        "inHazard": false,
        "sensorName": "HeadRoll"
      },
      {
        "inHazard": false,
        "sensorName": "HeadYaw"
      }
    ],
    "timeOfFlightSensorsHazardState": [
      {
        "inHazard": false,
        "sensorName": "TOF_Right"
      },
      {
        "inHazard": false,
        "sensorName": "TOF_Center"
      },
      {
        "inHazard": false,
        "sensorName": "TOF_Left"
      },
      {
        "inHazard": false,
        "sensorName": "TOF_Back"
      },
      {
        "inHazard": true,
        "sensorName": "TOF_DownFrontRight"
      },
      {
        "inHazard": false,
        "sensorName": "TOF_DownFrontLeft"
      },
      {
        "inHazard": false,
        "sensorName": "TOF_DownBackRight"
      },
      {
        "inHazard": true,
        "sensorName": "TOF_DownBackLeft"
      }
    ]
  }
}
```


JSON

Subscribing to HazardNotification Events

You can subscribe to `HazardNotification` event messages to receive notifications when Misty enters or exits a specific hazard state. You can use this information in your skill code to have Misty make decisions about how to move when autonomously navigating her environment.

The following code provides an example of registering for `HazardNotification` events in a skill running on Misty using her JavaScript API. In this code, we add return properties for `BumpSensorsHazardState` and `TimeOfFlightSensorsHazardState`. This allows us to parse `HazardNotification` event messages in a callback to understand which sensors are in a hazard state. This example uses `HazardNotification` messages to turn Misty's chest LED white when it's safe for her to drive, and red when Misty is in a hazard state. We populate an array with the names of the sensors that are in a hazard state, and we print that array to `SkillData` event listeners with the `misty.Debug()` method.

```
misty.Debug("HazardNotification skill starting!");

// White LED means it's safe to drive
// Red LED means Misty is in a hazard state

misty.ChangeLED(255, 255, 255);

misty.AddReturnProperty("Hazard", "BumpSensorsHazardState");
misty.AddReturnProperty("Hazard", "TimeOfFlightSensorsHazardState");
misty.RegisterEvent("Hazard", "HazardNotification", 0, true);

function _Hazard(data) {
    var safe = false;
    // Print HazardNotification event message & data from
    // added return properties
    misty.Debug(JSON.stringify(data));
    misty.Debug(JSON.stringify(data.AdditionalResults));
    const dataIn = data.AdditionalResults;
    // Push the name of each sensor that is in a hazard state
    // into an array called triggers
    var triggers = [];
    dataIn.forEach(sensor => {
        sensor.forEach(sensorData => {
            sensorData.InHazard ?
triggers.push(sensorData.SensorName) : {}
        });
    });
    // If the triggers array is empty, it's safe to drive.
    // If there are elements in this array, Misty is in
    // a hazard state.
    triggers.length ? {} : safe = true;
    safe ? misty.ChangeLED(255, 255, 255) : misty.ChangeLED(255, 0, 0);
    misty.Debug(safe);
    misty.Debug(triggers);
}
```

JavaScript

IMU

The IMU data stream provides information from Misty's Inertial Measurement Unit (IMU) sensor. It includes information about:

- the pitch, yaw, and roll orientation angles of the sensor (in degrees)
- the force (in meters per second) currently applied to the sensor along its pitch, yaw, and roll rotational axes
- the force (in meters per second squared) currently applied to the sensor along its X, Y, and Z axes

> **Note:** Misty's IMU orients its heading to 0/360 degrees each time Misty boots up or resets her real-time controller. For Misty, a `yaw` value of 0/360 degrees does **not** represent true north unless Misty is facing true north when the IMU orients its heading. Additionally, because the IMU is located in Misty's torso, readings from the

> IMU only change when Misty's body moves. They do not change relative to the position of Misty's head.

`IMU` message properties:

- `pitch` (double) - Misty's orientation on her pitch rotational axis (in degrees). A positive number means Misty is pitched at an angle forward, and a negative number means she is pitched back.
- `roll` (double) - Misty's orientation on her roll rotational axis (in degrees). A positive number means Misty is tipped at an angle to her right, and a negative number means she is tipped to her left.
- `yaw` (double) - Misty's orientation on her yaw rotational axis (in degrees). (Use this value to determine Misty's current heading when sending `DriveHead` commands.) A positive number means Misty is rotated to the left, and a negative number means she is rotated to the right.
- `pitchVelocity` (double) - The force (in meters per second) currently applied to Misty along her `pitch` rotational axis.
- `rollVelocity` (double) - The force (in meters per second) currently applied to Misty along her `roll` rotational axis.
- `yawVelocity` (double) - The force (in meters per second) currently applied to Misty along her `yaw` rotational axis.
- `xAcceleration` (double) - The force (in meters per second squared) currently applied to Misty along her `x` axis. A positive value means Misty is accelerating forward, and a negative value means she is accelerating backward.
- `yAcceleration` (double) - The force (in meters per second squared) currently applied to Misty along her `y` axis. A positive value means Misty is accelerating to her left, and a negative value means she is accelerating to her right.
- `zAcceleration` (double) - The force (in meters per second squared) currently applied to Misty along her `z` axis. A positive value means Misty is accelerating up, and a negative value means Misty is accelerating down. When Misty is set on a level surface, this value should be a negative number that indicates the force of gravity on Misty's IMU sensor.

```
IMU {
    "eventName": "IMU",
    "message": {
        "created": "2019-01-09T21:47:53.7607457Z",
        "expiry": "2019-01-09T21:47:54.1607457Z",
        "pitch":11.193,
        "pitchVelocity": 0.057,
        "roll": 2.468,
        "rollVelocity": -0.1,
        "sensorId": "imu",
        "sensorName": null,
        "xAcceleration": 0.75,
        "yAcceleration": 0.73,
        "yaw": 2.004,
        "yawVelocity": 0.096,
        "zAcceleration": 9.80
    },
    "type":"IMU"
}
```

JSON

By default, Misty sends `IMU` data to listeners of `IMU` events once every five seconds.

> **Tip:** Misty uses a **right-handed coordinate frame** to determine the value of each property returned in IMU event messages.

KeyPhraseRecognized

Misty sends `KeyPhraseRecognized` event messages when she recognizes the "Hey, Misty!" key phrase.

> **Note:** This event type is currently in **Beta**, and related hardware, firmware, or software is still under development. Feel free to use this event type, but recognize that it may behave unpredictably at this time.

`KeyPhraseRecognized` event messages include a `confidence` value that rates system's confidence in having heard the key phrase.

```
{
     "eventName":"KeyPhraseRecognizedEvent",
     "message": {
          "confidence":81,
          "created":"2019-07-31T06:53:18.151743Z"
     }
}'
```

JSON

Note

- When you call the `StartKeyPhraseRecognition` command, Misty listens for the key phrase by continuously sampling audio from the environment and comparing that audio to her trained key phrase model (in this case, "Hey, Misty!"). Misty does **not** create or save audio recordings while listening for the key phrase.
- To have Misty record what you say (for example, if you want to use speech to invoke other actions), you need to send a `StartRecordingAudio` command after receiving a `KeyPhraseRecognized` event. You can then do something with that audio file in your code, like hand it off to a third-party service for additional processing.
- Misty cannot record audio and listen for the "Hey, Misty!" key phrase at the same time. Sending a command to start recording audio automatically stops key phrase recognition. To have Misty start listening for the key phrase after recording an audio file, you must issue another `StartKeyPhraseRecognition` command.

You must start key phrase recognition before Misty can send `KeyPhraseRecognized` event messages. Follow these steps to code Misty to respond to the "Hey, Misty!" key phrase:

1. Invoke the `StartKeyPhraseRecognition` command.
2. Register for `KeyPhraseRecognized` events. When Misty hears the key phrase, she sends a message to `KeyPhraseRecognized` event listeners.
3. Write the code to handle what Misty should do when she hears the key phrase inside the `KeyPhrasedRecognized` event callback. For example, you might have Misty turn to face you or start recording audio to hand off to a third-party service for additional processing.

> **Note:** When Misty recognizes the key phrase, she automatically stops listening for key phrase events. In order to start Misty listening for the key phrase again, you need to issue another `StartKeyPhraseRecognition` command.

As an example of how to use this functionality in your skill code, the following has Misty play a sound and wave when she hears the key phrase.

```
StartKeyPhraseRecognition();

function StartKeyPhraseRecognition() {
    misty.Debug("Starting key phrase recognition...");
    // Starts Misty listening for the "Hey, Misty" key phrase
    misty.StartKeyPhraseRecognition();
    // Registers for KeyPhraseRecognized events
     misty.RegisterEvent("KeyPhraseRecognized","KeyPhraseRecognized", 10,
false);
     misty.Debug("KeyPhraseRecognition started. Misty will play a sound and
wave when she hears 'Hey Misty'.");
}

// Callback function to execute when Misty hears the key phrase
function _KeyPhraseRecognized() {
    waveRightArm();
    misty.Debug("Key phrase recognized!");
    misty.Debug("Audio recording stopped. Starting key phrase recognition
again...");
    // Starts Misty listening for the key phrase again
    StartKeyPhraseRecognition();
}

// Helper function to wave Misty's arm
function waveRightArm() {
    misty.MoveArmDegrees("left", 90, 45); // Left arm fully down
    misty.Pause(50);
    misty.MoveArmDegrees("right", 90, 45); // Right arm fully down
    misty.Pause(50); // Pause for 3 seconds
    misty.MoveArmDegrees("right", -45, 45); // Right arm fully up
    misty.Pause(7000); // Pause with arm up for 5 seconds (wave!)
    misty.MoveArmDegrees("right", 90, 45); // Right arm fully down
}
```

JavaScript

LocomotionCommand

`LocomotionCommand` WebSocket data is sent every time the linear or angular velocity of the robot changes. It is not sent at timed intervals.

Sample locomotion data:


```
LocomotionCommand{
     "EventName":"LocomotionCommand",
     "Message":{
          "ActionId":0,
          "AngularVelocity":0,
          "Created":"2018-04-02T22:59:39.3350238Z",
          "LinearVelocity":0.30000000000000004,
          "UsePid":true,
          "UseTrapezoidalDrive":true,
          "ValueIndex":0
     },
     "Type":"LocomotionCommand"
}
```


JavaScript

SelfState

The `SelfState` WebSocket provides a variety of data about Misty's current internal state, including:

- `battery` - Provides information about the state of Misty's battery, including charge percentage, voltage, and charging status.
- `cameraStatus` - Provides information about the status of Misty's RGB camera. This information can be useful when you need to check whether the RGB camera is ready or already in use before attempting to use it for another purpose.
- `localIPAddress` - Misty's current local IP address. Use this IP address to send Misty REST requests or set up WebSocket connections.
- `location` - A data object with information about Misty's current location and orientation ("pose") relative to her currently active occupancy grid.
- `robotRunState` - Provides information about Misty's current run state. Misty's `currentState` is `Loading` when the system is booting up; `Running` when the system has fully booted and is not updating; and `Updating` when the system is installing an update.
- `slamStatus` - The current status of Misty's simultaneous localization and mapping (SLAM) system. `SlamStatus` event messages can also be subscribed to independently. See the `SlamStatus` docs for more information.

Note: This event type is currently in **Alpha**, and related hardware, firmware, or software is still under development. Feel free to use this event type, but recognize that it may behave unpredictably at this time.

There are a number of fields in the event message data structure that are reserved for future use and which may be empty or contain `null` data:

- `Acceleration`
- `BumpedState`
- `CurrentGoal`
- `MentalState`
- `Personality`
- `PhysiologicalBehavior`

`SelfState` WebSocket messages are sent even if the data has not changed, as the data is sent via timed updates, instead of being triggered by events. The `SelfState` WebSocket can send data as frequently as every 100ms, though it is set by default to 250ms. To avoid having to handle excess data, you can change the message frequency for the WebSocket with the `DebounceMs` field, as shown in the `lightSocket.js` JavaScript helper.

Sample SelfState data:

SelfState


```
{
  "eventName": "SelfState",
  "message": {
    "acceleration": {
      "units": "None",
      "x": 0,
      "y": 0,
      "z": 0
    },
    "battery": {
      "chargePercent": null,
      "created": "2019-08-14T21:17:53.7266429Z",
      "current": -0.911,
      "expiry": "2019-08-14T21:18:03.7266429Z",
      "healthPercent": null,
      "isCharging": false,
```

```
    "sensorId": "charge",
    "sensorName": "/Sensors/RTC/BatteryCharge",
    "state": "Discharging",
    "temperature": 21,
    "trained": false,
    "voltage": 6.612
  },
  "bumpedState": {
    "disengagedSensorIds": [],
    "disengagedSensorNames": [],
    "disengagedSensors": [],
    "engagedSensorIds": [],
    "engagedSensorNames": [],
    "engagedSensors": []
  },
  "cameraStatus": {
    "isTakingPicture": false,
    "onboardCameraStatus": "Streaming"
  },
  "currentGoal": {
    "directedMotion": {
      "commandGroups": [
{
  "arguments": {
    "apiCommand": "MoveHead",
    "changeDisplayImageGesture": null,
    "lookParameters": {
      "actionDelayInSeconds": null,
      "actionDurationInSeconds": 1,
      "fractionalPupilUseLimit": null,
      "rollInRadians": -0.7330382858376184,
      "spatialPrecisionInMeters": null,
      "targetPose": {
        "bearing": 0,
        "created": "2019-08-14T20:28:48.3341954Z",
        "distance": 10,
        "elevation": 0,
        "frameId": "RobotNeck",
        "framesProvider": {
          "rootFrame": {
            "created": "2019-08-14T16:21:18.1035121Z",
            "id": "RobotBaseCenter",
            "isStatic": true,
            "linkFromParent": {
              "isStatic": true,
              "parentFrameId": "",
  "transformFromParent": {
    "bearing": 0,
    "distance": 0,
    "elevation": 0,
    "pitch": 0,
    "quaternion": {
      "isIdentity": true,
      "w": 1,
      "x": 0,
      "y": 0,
      "z": 0
    },
    "roll": 0,
    "x": 0,
    "y": 0,
    "yaw": 0,
    "z": 0
  },
  "transformToParent": {
    "bearing": 3.141592653589793,
    "distance": 0,
    "elevation": 0,
```

```
            "pitch": 0,
            "quaternion": {
              "isIdentity": true,
              "w": 1,
              "x": 0,
              "y": 0,
              "z": 0
            },
            "roll": 0,
            "x": 0,
            "y": 0,
            "yaw": 0,
            "z": 0
          }
        }
      }
    },
    "homogeneousCoordinates": {
      "bearing": 0,
      "distance": 10,
      "elevation": 0,
      "pitch": 0,
      "quaternion": {
        "isIdentity": true,
        "w": 1,
        "x": 0,
        "y": 0,
        "z": 0
      },
      "roll": 0,
      "x": 10,
      "y": 0,
      "yaw": 0,
      "z": 0
    },
    "pitch": 0,
    "roll": 0,
    "x": 10,
    "y": 0,
    "yaw": 0,
    "z": 0
  },
  "worldObjectId": null
 }
},
"name": "Misty2.AffectiveHeadEyeAction"
      }
    ],
    "id": 8835
  },
  "directedMotionBehavior": "Update",
  "haltActionSequence": false
},
"localIPAddress": "192.168.7.32",
"location": {
  "bearing": 0,
  "bearingThreshold": {
    "lowerBound": 0,
    "upperBound": 0
  },
  "distance": 0,
  "distanceThreshold": {
    "lowerBound": 0,
    "upperBound": 0
  },
  "elevation": 0,
  "elevationThreshold": {
    "lowerBound": 0,
```

```
      "upperBound": 0
    },
    "pose": {
      "bearing": 0,
      "created": "2019-08-14T21:17:53.9314769Z",
      "distance": 0,
      "elevation": 0,
      "frameId": "WorldOrigin",
      "framesProvider": {
        "rootFrame": {
          "created": "2019-08-14T16:21:18.1035121Z",
          "id": "RobotBaseCenter",
          "isStatic": true,
          "linkFromParent": {
            "isStatic": true,
            "parentFrameId": "",
            "transformFromParent": {
              "bearing": 0,
              "distance": 0,
              "elevation": 0,
              "pitch": 0,
              "quaternion": {
                "isIdentity": true,
                "w": 1,
                "x": 0,
                "y": 0,
                "z": 0
              },
              "roll": 0,
              "x": 0,
              "y": 0,
              "yaw": 0,
              "z": 0
            }
          }
        }
      },
      "transformToParent": {
        "bearing": 3.141592653589793,
        "distance": 0,
        "elevation": 0,
        "pitch": 0,
        "quaternion": {
          "isIdentity": true,
          "w": 1,
          "x": 0,
          "y": 0,
          "z": 0
        },
        "roll": 0,
        "x": 0,
        "y": 0,
        "yaw": 0,
        "z": 0
      }
    },
    "homogeneousCoordinates": {
      "bearing": 0,
      "distance": 0,
      "elevation": 0,
      "pitch": 0.10141706466674805,
      "quaternion": {
        "isIdentity": false,
        "w": 0.619889557,
        "x": -0.7773387,
        "y": -0.041995585,
        "z": 0.09861117
      },
      "roll": -1.7855754801963701,
```

```
            "x": 0,
            "y": 0,
            "yaw": 0.18964907113092544,
            "z": 0
          },
          "pitch": 0.10141706466674805,
          "roll": -1.7855754801963701,
          "x": 0,
          "y": 0,
          "yaw": 0.18964907113092544,
          "z": 0
        },
        "unitOfMeasure": "None"
      },
      "mentalState": {
        "affect": {
          "arousal": 0,
          "dominance": 0,
          "valence": 0
        },
        "created": "2019-08-14T21:17:53.9314769Z",
        "personality": {
          "agreeableness": 0,
          "conscientiousness": 0,
          "extraversion": 0,
          "neuroticism": 0,
          "openness": 0
        },
        "physiologicalBehavior": {
          "hunger": {
            "isEating": false,
            "level": 0
          },
          "sleepiness": {
            "isSleeping": false,
            "level": 0
          }
        }
      },
      "occupancyGridCell": {
        "x": 0,
        "y": 0
      },
      "occupancyGridCellMeters": 0,
      "serialMessages": [],
      "slamStatus": {
          "runMode": "Uninitialized",
          "sensorStatus": "Ready",
          "status": 2,
          "statusList": [
            "Ready"
          ]
        },
        "touchedState": {
          "disengagedSensors": [],
          "engagedSensors": []
        }
    }
  }
```


JSON

SerialMessage

The `SerialMessage` data stream provides information sent to Misty by external hardware connected to the ports on her back. `SerialMessage` events trigger when Misty receives data sent through one of these ports.

To send `SerialMessage` data to Misty from an Arduino, use:

```
// ARDUINO

Serial.println("<your_data_to_Misty>")
```

C

Sending data to Misty as a JSON string can make it easier to parse the data in your skill code. This example shows how you can format data from a temperature and pressure sensor connected to an Arduino:

```
// ARDUINO

Serial.println("
{\"temperature\":\""+String(<temp_value>)+"\",\"pressure\":\""+String(<pres
sure_value>)+"\"}");
```

C

Handle this data with Misty's on-robot JavaScript API by registering for `SerialMessage` events. Add `SerialMessage` as an additional return property, and parse the data in the `_SerialMessage()` callback that triggers when the data is ready.

```
// MISTY

// Register for SerialMessage events and add SerialMessage as a return
property
misty.AddReturnProperty("SerialMessage", "SerialMessage");
misty.RegisterEvent("SerialMessage", "SerialMessage", 0, true);

function _SerialMessage(data) {
    if(data !== undefined && data !== null) {
        // Parse SerialMessage data and assign it to a variable
        var obj = JSON.parse(data.AdditionalResults[0].Message);
        var temp = obj.temperature;
        var pressure = obj.pressure;
    }
}
```

JavaScript

For more about events and callbacks, see the Data Handling: Events & Callbacks section of JavaScript SDK Architecture.

SkillData

Subscribe to the `SkillData` named object to see debug messages, error messages, and other data JavaScript skills publish during skill execution. Use the `misty.Debug()` command in a skill to send a `SkillData` message.

The value of the `BroadcastMode` parameter in a skill's .json meta file determines when the skill sends `SkillData` messages, and what kind of data those messages include.

- `Off` - The skill does not send `SkillData` messages.
- `Debug` - The skill prints error and debug messages to `SkillData` events.
- `Verbose` - In addition to error and debug messages, the skill sends a message for each command that Misty receives to `SkillData` events.

When you connect Misty to the Skill Runner to start and stop skills, the web page subscribes to `SkillData` events and skill messages print to the console in your web browser. You can create your own subscription to `SkillData` messages by connecting to Misty's WebSocket server.

SkillData Message Examples

This sample shows the `SkillData` message sent when a skill executes the `misty.Debug()` command with the string `"Hello, world!"`:

```
{
  "eventName": "SkillData",
  "message": {
    "data": {
      "data": "Hello, world!"
    },
    "message": "Calling command 'Debug'",
    "timestamp": "2019-04-28T22:29:33.0441867Z",
    "truncated": false
  }
}
```

JSON

This sample shows the `SkillData` message sent when a skill executes a `misty.SendExternalRequest()` command. The `message.data` object provides information about the values passed in for the command's arguments.

```
{
  "eventName": "SkillData",
  "message": {
    "data": {
      "method": "GET",
      "resource": "http://soundbible.com/grab.php?id=1949&type=mp3",
      "authorizationType": "null",
      "token": "null",
      "arguments": "null",
      "save": true,
      "apply": true,
      "fileName": "sound",
      "callback": "null",
      "callbackRule": "null",
      "skillToCall": "null",
      "prePauseMs": 0,
      "postPauseMs": 0
    },
    "message": "Calling command 'SendExternalRequest'",
    "timestamp": "2019-04-28T22:29:33.0285627Z",
    "truncated": false
  }
}
```

JSON

Subscribing to SkillData Events

To subscribe to `SkillData` events, you open a WebSocket connection to Misty and send a subscription message to the `SkillData` named object.

This example shows how to subscribe to `SkillData` events in a web page using the WebSocket API. In the example, `SkillData` messages print to the web browser's console.

```
//Misty's IP address
const ip = "<robot-ip-address>";

function streamSkillData() {
    //Open a WebSocket connection to Misty
    const ws = new WebSocket("ws://" + ip + "/pubsub");

    //Send a message to subscribe to SkillData events
    ws.onopen = function(event) {
        ws.send(JSON.stringify(
```

```
                {
                "Operation": "subscribe",
                "Type": "SkillData",
                "DebounceMs": null,
                "EventName": "SkillData",
                "Message": "",
                "ReturnProperty": null
                }
            ));
        }

        //Parse and log SkillData messages
        ws.onmessage = function(event) {
            var data = event.data
            console.log(data);
        };
    };

    streamSkillData();
```

JavaScript

Before you close the WebSocket connection, send a message to unsubscribe to `SkillData` events. You cannot set up a `SkillData` subscription when there is an open subscription with the same `EventName` you are trying to subscribe to.

```
ws.send(JSON.stringify(
    {
    "Operation": "unsubscribe",
    "EventName": "SkillData",
    "Message": ""
    }
));
ws.close();
```

JavaScript

SlamStatus

Misty's `SlamStatus` event messages provide information about the current status of Misty's simultaneous localization and mapping (SLAM) system.

> **Note:** Misty's SLAM capabilities are an alpha feature. Experiment with mapping, but recognize that Misty's ability to create maps and track within them is unreliable at this time.

`SlamStatus` messages include the following properties:

- `status` (int) - Number that describes the current status of the SLAM system. This number updates with information from the `sensorStatus` and `runMode` fields, as well as with other events that occur during a SLAM session. Note that this number represents several status codes simultaneously. You can convert this number to a binary value to see whether the bit field for a given status code is on (`1`) or off (`0`). As an example, the status code `33028` converts to a binary value of `1000000100000100`. In this binary value, the 3rd, 9th, and 16th bits are flipped. Those bits correspond to the status codes for `Exploring`, `LostPose`, and `Streaming`, respectively. (Note that the system also returns the string fields for all current status codes to the `statusList` array that comes back with a `SlamStatus` event message.) The following hexadecimal values correspond to bit fields for each possible status code:
    - 0x0000: `Uninitialized` - The SLAM system is not yet initialized.
    - 0x0001: `Initializing` - The SLAM system is initializing.

    - 0x0002: `Ready` - Misty's depth sensor and the SLAM system are ready to start mapping and tracking.
    - 0x0004: `Exploring` - The SLAM system is mapping.
    - 0x0008: `Tracking` - The SLAM system is tracking.
    - 0x0010: `Recording` - The SLAM system is recording an `.occ` file to Misty's local storage.
    - 0x0020: `Resetting` - The SLAM system is in the process of shutting down and resetting.
    - 0x0040: `Rebooting` - The SLAM system is rebooting.
    - 0x0080: `HasPose` - The SLAM system has obtained pose.
    - 0x0100: `LostPose` - The SLAM system has lost pose after having obtained it.
    - 0x0200: `Exporting_Map` - The SLAM system is exporting a map after mapping is complete.
    - 0x0400: `Error` - There is an error with the SLAM system or with the depth sensor.
    - 0x0800: `Error_Sensor_Not_Connected` - The depth sensor is not connected.
    - 0x1000: `Error_Sensor_No_Permission` - The system does not have permission to use the depth sensor.
    - 0x2000: `Error_Sensor_Cant_Open` - The system cannot open the depth sensor for communication.
    - 0x4000: `Error_Error_Power_Down_Robot` - Unrecoverable error. Power down the robot and restart.
    - 0x8000: `Streaming` - The SLAM system is streaming.
- `statusList` (array) - A list of the string values that describe the current status of the SLAM system. Can contain any of the values represented by the `status` field.
- `runMode` (string) - Current status of the navigation system. Possible values are:
    - `Uninitialized`
    - `Tracking`
    - `Exploring`
    - `Relocalizing`
    - `Paused`
    - `ExportingScene`
    - `NeedMoreMotionToInitMap`
    - `NotAvailable`
- `sensorStatus` (string) - Current status of the depth sensor sensor. Possible values are:
    - `Uninitialized`
    - `Connected`
    - `Booting`
    - `Ready`
    - `Disconnected`
    - `Error`
    - `USBError`
    - `LowPowerMode`
    - `RecoveryMode`
    - `ProdDataCorrupt`
    - `CalibMissingOrInvalid`
    - `FWVersionMismatch`
    - `FWUpdate`
    - `FWUpdateComplete`
    - `FWUpdateFailed`
    - `FWCorrupt`
    - `EndOfFile`
    - `USBDriverNotInstalled`
    - `Streaming`

> **Note:** We suggest primarily using the values of `Status` / `StatusList` when coding SLAM functionality in your skills and robot applications, and only using the `SensorStatus` and `RunMode` values as supplemental information if needed or for debugging purposes.

Example `SlamStatus` event message:

```
{
     "eventName":"SlamStatus",
     "message": {
          "slamStatus": {
               "runMode":"Uninitialized",
               "sensorStatus":"Connected",
               "status":2,
               "statusList":["Ready"]
          }
     }
}
```

JSON

SourceTrackDataMessage

The `SourceTrackDataMessage` named object provides information about the location and volume of the noise or spoken voice that Misty can detect.

> **Note:** Misty only sends audio localization event messages when she is actively recording audio. When using audio localization data in your skills, you need to call the `misty.StartRecordingAudio()` method in order to receive `SourceTrackDataMessage` or `SourceFocusConfigMessage` event messages.
>
> This event type is currently in **Beta**, and related hardware, firmware, or software is still under development. Feel free to use this event type, but recognize that it may behave unpredictably at this time.

Audio localization messages include the following data:

- `DegreeOfArrivalNoise` : An array where each element is a value between 0-360 that indicates the angle of arrival for a noise that Misty detects. This array can contain the degree of arrival noise for up to three unique sounds. Each value in this array represents the degree of arrival for a single sound.
- `DegreeOfArrivalSpeech` : A value between 0-360 that indicates the angle relative to Misty where she detected the loudest voice.
- `TimeOffset` : The time (in milliseconds) that Misty captured the audio sample relative to the start time of the audio recording. Misty records samples at roughly 20hz, or once every 50ms.
- `VoiceActivityPolar` : A 360 element array where each element is a value between 0 and 255 that indicates the level of sound activity detected at a particular angle. The higher the value, the more voice activity detected.
- `VoiceActivitySectors` : A four element array of boolean values indicating whether Misty detected voice activity in a particular sector. Each sector represents a 90 degree wedge of the area surrounding Misty. The front-facing wedge is offset, with Misty's face pointing toward the 45 degree angle down its center.

Sample `SourceTrackDataMessage` response data:

```
{
  "eventName": "SourceTrackDataMessageEvent",
  "message": {
```

```
    "degreeOfArrivalNoise": [90],
    "degreeOfArrivalSpeech": 360,
    "timeOffset": 4925,
    "voiceActivityPolar":
[6,6,6,5,5,5,5,5,5,4,4,3,3,3,2,2,2,2,2,2,2,2,2,2,2,3,3,3,3,3,3,3,3,4,4,4,
3,3,2,2,3,5,6,5,4,4,4,5,6,6,6,7,7,8,8,9,9,9,9,9,10,10,10,10,10,10,9,9,9,10,10
,10,9,8,7,7,7,7,8,8,8,7,7,7,6,6,5,5,5,4,4,4,4,3,3,3,2,1,1,0,0,1,1,1,1,1,1,1,1
,1,1,1,1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
0,0,0,0,1,1,2,3,3,4,4,5,5,6,6,7,8,8,8,9,9,9,9,9,9,9,8,8,8,8,7,7,6,6,5,5,3,2
,1,3,4,4,3,2,1,1,2,3,3,4,4,4,4,3,2,1,2,2,3,3,4,4,3,3,2,4,5,5,4,4,4,5,5,5,6,5,
5,5,5,4,4,5,5,4,3,3,4,5,5,5,4,4,4,4,5,5,4,3,2,1,1,1,1,1,2,2,2,3,3,2,2,2,2,2
,2,3,3,3,3,3,3,3,3,4,4,4,4,4,4,5,5,5,5,5,5,5,5,5,5,5,5,4,4,3,1,1,1,1,1,1,0,0,
0,0,0,0,0,0,0,0,0,0,0,0,0,0,1,1,1,1,2,3,3,4,5,5,5,6,6,7,8,8,8,8,8,8,8,9,9,9,9
,8,8,8,7,7,7,7,7,6,6,6,6,6,6,6,6,6,6,6],
    "voiceActivitySectors": [false, false,false, false ]
  }
}
```


JSON

SourceFocusConfigMessage

The `SourceFocusConfigMessage` named object provides meta information about the configuration of audio localization data. The system only sends this message once, when Misty starts recording audio.

> **Note:** Misty only sends `SourceFocusConfigMessage` data once, when she starts recording audio. When using audio localization data in your skills, you need to call the `misty.StartRecordingAudio()` method in order to receive `SourceTrackDataMessage` or `SourceFocusConfigMessage` event messages.
>
> This event type is currently in **Beta**, and related hardware, firmware, or software is still under development. Feel free to use this event type, but recognize that it may behave unpredictably at this time.

- `GainStep` : A value that indicates the spatial gain applied to the sector where Misty detects a voice. You can safely ignore this value.
- `SectorsEnabled` : A four element array of boolean values that indicate whether audio localization is enabled for a given sector. At this time, all sectors are enabled be default and cannot be disabled.
- `SectorStartAngles` : A four element array of integer values between 0 - 360 indicating the angle bounding the start of a given sector. Each sector represents a 90 degree wedge of the environment that surrounds Misty. Each sector stops at the angle where the next sector begins. These sectors correspond to the `VoiceActivitySectors` array in the `SourceTrackDataMessage` named object.

Sample `SourceFocusConfigMessage` response data:


```
{
  "eventName": "SourceFocusConfigMessageEvent",
  "message": {
    "gainStep": 65535,
    "sectorsEnabled": [true, true, true, true],
    "sectorStartAngles": [135,45,315,225]
  }
}
```


JSON

TimeOfFlight

Misty has four edge and four range time-of-flight sensors that provide a single stream of raw proximity data. These sensors send `TimeOfFlight` messages that you can subscribe to in your skills and robot applications.

> You can change how frequently a `TimeOfFlight` subscription gets messages by adjusting the value of the `debounceMS` parameter when you subscribe to the WebSocket connection or call the `misty.RegisterEvent()` method in Misty's JavaScript API.

A `TimeOfFlight` event message includes the following key/value pairs:

- `created` - A timestamp that tells us when the system created the message.
- `distanceInMeters` - The distance (in meters) of the nearest object the sensor detects.
- `sensorId` - An identification string associated with a specific time-of-flight sensor. In `TimeOfFlight` events, the value of `sensorId` tells us which sensor sent the message. See the Time-of-Flight Sensor Details table for a list of `sensorId` values and their corresponding time-of-flight sensors.
- `status` - A status code from the sensor that provides additional context for the distance reading in a `TimeOfFlight` event message. At the macro level, `0` indicates the distance reading is valid; `100` codes are warnings (the system has low confidence in the value); and `200` codes are errors (the distance value is not reliable and should probably be discarded). Status codes are different for edge and range time-of-flight sensors. For a list of status codes and their descriptions, see the Time-of-Flight Status Codes section below.
- `type` - A string value indicating whether this message comes from a `Range` or `Edge` time-of-flight sensor. See the Time-of-Flight Sensor Details table for a list of which sensors belong in each category.

```
// Example message from right front ToF sensor
{
  "eventName": "TimeOfFlight",
  "message": {
    "created": "2019-08-22T20:29:30.7786174Z",
    "distanceInMeters": 0.19,
    "sensorId": "toffr",
    "status": 0,
    "type": "Range"
  }
}
```

JSON

Time-of-Flight Sensor Details

Sensor Position	sensorName	sensorId	type
Right Front	toF_Right	toffr	Range
Left Front	toF_Left	toffl	Range
Center Front	toF_Center	toffc	Range
Back	toF_Back	tofr	Range
Downward Front Right	toF_DownFrontRight	tofdfr	Edge
Downward Front Left	toF_DownFrontLeft	tofdfl	Edge
Downward Back Right	toF_DownBackRight	tofdrr	Edge
Downward Back Left	toF_DownBackLeft	tofdrl	Edge

Time-of-Flight Status Codes

Range Sensor Status Codes

Event messages from Misty's range time-of-flight sensors include one of the following status codes:

Status Code	Name	Meaning
`-1`	`Unknown`	An unknown error. The system uses this code as a catch-all for errors that do not map to one of the codes below.
`0`	`RangeValid`	The distance reading is valid. The sensor received good data, and its readings fell within the thresholds set for both sigma and signal.
`101`	`SigmaFail`	The distance reading may not be accurate. The system sends this code when the standard deviation (sigma) of the sensor's readings is higher than the threshold set in Misty's firmware. You are more likely to see this code when Misty is operating in bright environments, or when the sensor points toward a flat surface at an angle.
`102`	`SignalFail`	The distance reading may not be accurate. The system sends this code when the return signal is too weak for the sensor to provide good data. You are more likely to see this code when the target of a sensor is too far away, not reflective enough, or too small.
`103`	`ROIOutOfBounds`	Each range time-of-flight sensor is comprised of a 16x16 array of detectors. The region of interest for each sensor can be adjusted to allow a different area of the sensor to read return signals. This allows us to change which direction the sensor "sees". (This is similar to the way that you can move your eyes to look in different directions, albeit with more restricted movement.) If the chosen region of interest is outside the 16x16 array of detectors, the system returns a distance value of 8191 with a `103` status code. The region of interest is currently set internally on Misty's system and cannot be adjusted. You will not see this error until the ability to change the region of interest is exposed.
`104`	`OutOfBounds`	The distance reading may not be accurate. The system sends this code when the sensor sees a target further away than 1.2 meters. While Misty's range time-of-flight sensors **do** return distance values for targets further away than 1.2 meters, the sensor's proximity calculations are less accurate when the target is outside this range. Bear this mind when using distance values that come back with this status code.
`207`	`WrapTargetFail`	The distance reading is not accurate due to range aliasing. This can happen when the sensor's target is highly reflective and outside the sensor's measurable range. This error can cause the sensor to read the distance of an object further than ~1.3 meters as being within range of the sensor. When you receive a `207` error, you can calculate the approximate distance of the target by adding 1.3 meters to the measured distance that Misty returns.

Edge Sensor Status Codes

Event messages from Misty's range time-of-flight sensors include one of the following status codes:

Status Code	Name	Meaning
`-1`	`Unknown`	An unknown error. The system uses this code as a catch-all for errors that do not map to one of the codes below.
`0`	`RangeValid`	The distance reading is valid. The sensor received good data, and its readings fell within the thresholds set for both sigma and signal.
`201`	`HardwareFail`	The sensor failed to initialize due to an internal error. Because the system automatically attempts to reinitialize the sensors, and because these sensors do not send messages until they are fully initialized, it is unlikely that you will see

Status Code	Name	Meaning
		this code. Errors `201 - 205` only happen at time of initialization.
202	HardwareFail2	The sensor failed to initialize due to an internal error. Because the system automatically attempts to reinitialize the sensors, and because these sensors do not send messages until they are fully initialized, it is unlikely that you will see this code. Errors `201 - 205` only happen at time of initialization.
203	HardwareFail3	The sensor failed to initialize due to an internal error. Because the system automatically attempts to reinitialize the sensors, and because these sensors do not send messages until they are fully initialized, it is unlikely that you will see this code. Errors `201 - 205` only happen at time of initialization.
204	HardwareFail4	The sensor failed to initialize due to an internal error. Because the system automatically attempts to reinitialize the sensors, and because these sensors do not send messages until they are fully initialized, it is unlikely that you will see this code. Errors `201 - 205` only happen at time of initialization.
205	HardwareFail5	The sensor failed to initialize due to an internal error. Because the system automatically attempts to reinitialize the sensors, and because these sensors do not send messages until they are fully initialized, it is unlikely that you will see this code. Errors `201 - 205` only happen at time of initialization.
206	EarlyConvergence	The sensor has not received a minimum number of return signals within 0.5 milliseconds. When this happens, the sensor cancels measurement to conserve power and returns this status code. You are most likely to see this code when there is nothing in view of the sensor within its range of detection.
207	MaxConvergence	The sensor failed to detect a valid target after the maximum amount of time allowed for convergence.
208	RangeIgnore	The number of returned signals is less than the threshold set in Misty's firmware, AND the measured distance of any returned signals is less than the crosstalk distance (7 mm) set in the firmware. You should only see this message if there are no solid objects within the sensor's range, causing the sensor to read its protective lens as the strongest signal. You might see this error if the sensor is over an edge, picked up in the air, or if Misty is tipped over.
211	Signal-To-NoiseRatioFail	Too much ambient light is washing out the sensor's returned signals. You are most likely to see this error when shining a light directly into the sensor.
212	RawRangeUnderflow	The system has produced a negative value for the proximity of the nearest target. This typically happens when the target is closer than the offset distance the system uses in proximity calculations (or, in other words, if the target is closer than 10mm). You might see this error if you place your finger on the sensor's lens, or when the sensor is measuring the distance of bright target between 1.2 and 1.5 meters away.
213	OutOfBounds	The sensor's target is too far away and the system cannot calculate an accurate distance value. You should only see this status when the sensor's target is more than 200mm away from the sensor.
214	RangingUnderflow	The system has produced a negative value for the proximity of the nearest target. This typically happens when the target is closer than the offset distance the system uses in proximity calculations (or, in other words, if the target is closer than 10mm). You might see this error if you place

Status Code	Name	Meaning
		your finger on the sensor's lens, or when the sensor is measuring the distance of bright target between 1.2 and 1.5 meters away.

TouchSensor

The `TouchSensor` data stream sends information each time one of the capacitive touch sensors on Misty's head is touched or released. In the `TouchSensor` data object, the value of the `sensorName` property is one of the following strings, indicating which sensor triggered the event:

- `CapTouch_Chin`
- `CapTouch_HeadLeft`
- `CapTouch_HeadRight`
- `CapTouch_HeadBack`
- `CapTouch_HeadFront`
- `CapTouch_Scruff`

The value of the `sensorPosition` property is a string indicating the location of the sensor. This value corresponds with the name of the sensor that triggered the event (i.e. when the `CapTouch_Chin` sensor triggers the event, the value of `sensorPosition` is `Chin`). The value of `isContacted` is a boolean indicating whether the sensor was touched (`true`) or released (`false`). The `TouchSensor` data object also provides “pose” information about Misty at the time of the event. For more about pose, see the mapping section of the Command Center documentation.

Sample `TouchSensor` data:


```
TouchSensor{
    "eventName":"TouchSensor",
    "message":{
       "created":"2019-01-09T22:55:20.5300385Z",
       "expiry":"2019-01-09T22:55:21.5300385Z",
       "isContacted":true,
       "pose":{
          "bearing":1.0303768429594167,
          "created":"2019-01-09T20:11:03.2951639Z",
          "distance":0.09899494809418499,
          "elevation":-0.9409567798080669,
          "frameId":"RobotHead",
          "framesProvider":{
             "rootFrame":{
                "created":"2019-01-09T20:11:03.2013843Z",
                "id":"RobotBaseCenter",
                "isStatic":true,
                "linkFromParent":{
                   "isStatic":true,
                   "parentFrameId":"",
                   "transformFromParent":{
                      "bearing":0,
                      "distance":0,
                      "elevation":0,
                      "pitch":0,
                      "quaternion":{
                         "isIdentity":true,
                         "w":1,
                         "x":0,
                         "y":0,
                         "z":0
                      },
                      "roll":0,
```

```
            "x":0,
            "y":0,
            "yaw":0,
            "z":0
        },
        "transformToParent":{
            "bearing":3.141592653589793,
            "distance":0,
            "elevation":0,
            "pitch":0,
            "quaternion":{
                "isIdentity":true,
                "w":1,
                "x":0,
                "y":0,
                "z":0
            },
            "roll":0,
            "x":0,
            "y":0,
            "yaw":0,
            "z":0
        }
    },
    "homogeneousCoordinates":{
        "bearing":1.0303768429594167,
        "distance":0.09899494809418499,
        "elevation":-0.9409567798080669,
        "pitch":-1.399999976158142,
        "quaternion":{
            "isIdentity":false,
            "w":0.7648422,
            "x":0,
            "y":-0.64421767,
            "z":0
        },
        "roll":0,
        "x":0.029999999329447746,
        "y":0.05000000074505806,
        "yaw":0,
        "z":0.07999999821186066
    },
    "pitch":-1.399999976158142,
    "roll":0,
                "x":0.029999999329447746,
                "y":0.05000000074505806,
                "yaw":0,
                "z":0.07999999821186066
            },
            "sensorId":"cap",
            "sensorName":"CapTouch_HeadLeft",
            "sensorPosition":"HeadLeft"
        },
        "type":"TouchSensor"
}
```


JSON

WorldState

The `WorldState` named object exists to provide information about things Misty perceives in her environment, including:

- the locations of perceived objects

- the times they were perceived

> **Note:** The `WorldState` event type is currently in **Alpha** and can't be used for much at this time. Many of the properties Misty sends in `WorldState` event messages are reserved for future use, and may have `null` or invalid values.


```
{
  "eventName": "WorldState",
  "message": {
    "characteristics": [],
    "objects": [
      {
        "class": null,
        "id": 1,
        "individual": null,
        "isSensible": true,
        "location": {
          "bearing": -0.5584253393087596,
          "bearingThreshold": {
            "lowerBound": 0,
            "upperBound": 0
          },
          "distance": 0.15293597670901554,
          "distanceThreshold": {
            "lowerBound": 0,
            "upperBound": 0
          },
          "elevation": 0.38783942844582714,
          "elevationThreshold": {
            "lowerBound": 0,
            "upperBound": 0
          },
          "pose": {
            "bearing": -0.5584253393087596,
            "created": "2019-09-09T19:15:10.7842637Z",
            "distance": 0.15293597670901554,
            "elevation": 0.38783942844582714,
            "frameId": "WorldOrigin",
            "framesProvider": {
              "rootFrame": {
                "created": "2019-09-09T18:30:21.4398142Z",
                "id": "RobotBaseCenter",
                "isStatic": true,
                "linkFromParent": {
                  "isStatic": true,
                  "parentFrameId": "",
                  "transformFromParent": {
                    "bearing": 0,
                    "distance": 0,
                    "elevation": 0,
                    "pitch": 0,
                    "quaternion": {
                      "isIdentity": true,
                      "w": 1,
                      "x": 0,
                      "y": 0,
                      "z": 0
                    },
                    "roll": 0,
                    "x": 0,
                    "y": 0,
                    "yaw": 0,
                    "z": 0
                  },
```

```
          "transformToParent": {
            "bearing": 3.141592653589793,
            "distance": 0,
            "elevation": 0,
            "pitch": 0,
            "quaternion": {
              "isIdentity": true,
              "w": 1,
              "x": 0,
              "y": 0,
              "z": 0
            },
            "roll": 0,
            "x": 0,
            "y": 0,
            "yaw": 0,
            "z": 0
          }
        }
      }
    },
    "homogeneousCoordinates": {
      "bearing": -0.5584253393087596,
      "distance": 0.15293597670901554,
      "elevation": 0.38783942844582714,
      "pitch": 1.5696512460708618,
      "quaternion": {
        "isIdentity": false,
        "w": 0.7075245,
        "x": -0.0000965454237,
        "y": 0.7066889,
        "z": -0.0000882303138
      },
      "roll": -0.21764267965049405,
      "x": 0.12007024884223938,
      "y": -0.0750148594379425,
      "yaw": -0.21763145560213495,
      "z": -0.05783873423933983
    },
    "pitch": 1.5696512460708618,
    "roll": -0.21764267965049405,
    "x": 0.12007024884223938,
    "y": -0.0750148594379425,
    "yaw": -0.21763145560213495,
    "z": -0.05783873423933983
  },
  "unitOfMeasure": "None"
},
"statistics": {
  "durationContinuouslyPerceived": "00:00:00",
  "firstPerceived": "2019-09-09T19:15:08.9239249Z",
  "mostRecentlyPerceived": "2019-09-09T19:15:10.7842637Z"
},
"timeStamp": "2019-09-09T19:15:10.7842637Z"
},
{
  "class": null,
  "id": 2,
  "individual": null,
  "isSensible": true,
  "location": {
    "bearing": -2.4473488655806275,
    "bearingThreshold": {
      "lowerBound": 0,
      "upperBound": 0
    },
    "distance": 0.1320835387735192,
    "distanceThreshold": {
```

```
      "lowerBound": 0,
      "upperBound": 0
    },
    "elevation": -0.4793232003535818,
    "elevationThreshold": {
      "lowerBound": 0,
      "upperBound": 0
    },
    "pose": {
      "bearing": -2.4473488655806275,
      "created": "2019-09-09T19:15:10.7842637Z",
      "distance": 0.1320835387735192,
      "elevation": -0.4793232003535818,
      "frameId": "WorldOrigin",
      "framesProvider": {
        "rootFrame": {
          "created": "2019-09-09T18:30:21.4398142Z",
          "id": "RobotBaseCenter",
          "isStatic": true,
          "linkFromParent": {
            "isStatic": true,
            "parentFrameId": "",
            "transformFromParent": {
              "bearing": 0,
              "distance": 0,
              "elevation": 0,
              "pitch": 0,
              "quaternion": {
                "isIdentity": true,
                "w": 1,
                "x": 0,
                "y": 0,
                "z": 0
              },
              "roll": 0,
              "x": 0,
              "y": 0,
              "yaw": 0,
              "z": 0
            },
            "transformToParent": {
              "bearing": 3.141592653589793,
              "distance": 0,
              "elevation": 0,
              "pitch": 0,
              "quaternion": {
                "isIdentity": true,
                "w": 1,
                "x": 0,
                "y": 0,
                "z": 0
              },
              "roll": 0,
              "x": 0,
              "y": 0,
              "yaw": 0,
              "z": 0
            }
          }
        }
      },
      "homogeneousCoordinates": {
        "bearing": -2.4473488655806275,
        "distance": 0.1320835387735192,
        "elevation": -0.4793232003535818,
        "pitch": -1.5696512460708618,
        "quaternion": {
          "isIdentity": false,
```

```
            "w": 0.00008819939,
            "x": 0.7066889,
            "y": 0.00009651453,
            "z": 0.7075245
          },
          "roll": 0.21757214098211117,
          "x": -0.09007161110639572,
          "y": -0.07498423755168915,
          "yaw": 2.924042367538678,
          "z": 0.06091412156820297
        },
        "pitch": -1.5696512460708618,
        "roll": 0.21757214098211117,
        "x": -0.09007161110639572,
        "y": -0.07498423755168915,
        "yaw": 2.924042367538678,
        "z": 0.06091412156820297
      },
      "unitOfMeasure": "None"
    },
    "statistics": {
      "durationContinuouslyPerceived": "00:00:00",
      "firstPerceived": "2019-09-09T19:15:08.9239249Z",
      "mostRecentlyPerceived": "2019-09-09T19:15:10.7842637Z"
    },
    "timeStamp": "2019-09-09T19:15:10.7842637Z"
  },
  {
    "class": null,
    "id": 3,
    "individual": null,
    "isSensible": true,
    "location": {
      "bearing": 0.5579833753366954,
      "bearingThreshold": {
        "lowerBound": 0,
        "upperBound": 0
      },
      "distance": 0.17502626966973778,
      "distanceThreshold": {
        "lowerBound": 0,
        "upperBound": 0
      },
      "elevation": 0.6283228116298827,
      "elevationThreshold": {
        "lowerBound": 0,
        "upperBound": 0
      },
      "pose": {
        "bearing": 0.5579833753366954,
        "created": "2019-09-09T19:15:10.7842637Z",
        "distance": 0.17502626966973778,
        "elevation": 0.6283228116298827,
        "frameId": "WorldOrigin",
        "framesProvider": {
          "rootFrame": {
            "created": "2019-09-09T18:30:21.4398142Z",
            "id": "RobotBaseCenter",
            "isStatic": true,
            "linkFromParent": {
              "isStatic": true,
              "parentFrameId": "",
              "transformFromParent": {
                "bearing": 0,
                "distance": 0,
                "elevation": 0,
                "pitch": 0,
                "quaternion": {
```

```
        "isIdentity": true,
        "w": 1,
        "x": 0,
        "y": 0,
        "z": 0
      },
      "roll": 0,
      "x": 0,
      "y": 0,
      "yaw": 0,
      "z": 0
    },
    "transformToParent": {
      "bearing": 3.141592653589793,
      "distance": 0,
      "elevation": 0,
      "pitch": 0,
      "quaternion": {
        "isIdentity": true,
        "w": 1,
        "x": 0,
        "y": 0,
        "z": 0
      },
      "roll": 0,
      "x": 0,
      "y": 0,
      "yaw": 0,
      "z": 0
    }
  }
},
"homogeneousCoordinates": {
  "bearing": 0.5579833753366954,
  "distance": 0.17502626966973778,
  "elevation": 0.6283228116298827,
  "pitch": 1.5696512460708618,
  "quaternion": {
    "isIdentity": false,
    "w": 0.7075245,
    "x": -0.0000965454237,
    "y": 0.7066889,
    "z": -0.0000882303138
  },
  "roll": -0.21764267965049405,
  "x": 0.12012173235416412,
  "y": 0.07497323304414749,
  "yaw": -0.21763145560213495,
  "z": -0.10287846624851227
},
"unitOfMeasure": "None"
  },
  "statistics": {
    "durationContinuouslyPerceived": "00:00:00",
    "firstPerceived": "2019-09-09T19:15:08.9239249Z",
    "mostRecentlyPerceived": "2019-09-09T19:15:10.7842637Z"
  },
  "timeStamp": "2019-09-09T19:15:10.7842637Z"
},
{
```

```
"class": null,
"id": 4,
"individual": null,
"isSensible": true,
"location": {
  "bearing": -3.1415883669408133,
  "bearingThreshold": {
    "lowerBound": 0,
    "upperBound": 0
  },
  "distance": 0.4359997212269757,
  "distanceThreshold": {
    "lowerBound": 0,
    "upperBound": 0
  },
  "elevation": 0.0011690930720143118,
  "elevationThreshold": {
    "lowerBound": 0,
    "upperBound": 0
  },
  "pose": {
    "bearing": -3.1415883669408133,
"created": "2019-09-09T19:15:10.7358941Z",
"distance": 0.4359997212269757,
"elevation": 0.0011690930720143118,
"frameId": "WorldOrigin",
"framesProvider": {
  "rootFrame": {
    "created": "2019-09-09T18:30:21.4398142Z",
    "id": "RobotBaseCenter",
    "isStatic": true,
    "linkFromParent": {
      "isStatic": true,
      "parentFrameId": "",
      "transformFromParent": {
        "bearing": 0,
        "distance": 0,
        "elevation": 0,
        "pitch": 0,
        "quaternion": {
          "isIdentity": true,
          "w": 1,
          "x": 0,
          "y": 0,
          "z": 0
        },
        "roll": 0,
        "x": 0,
        "y": 0,
        "yaw": 0,
        "z": 0
      },
      "transformToParent": {
        "bearing": 3.141592653589793,
        "distance": 0,
        "elevation": 0,
        "pitch": 0,
        "quaternion": {
          "isIdentity": true,
          "w": 1,
          "x": 0,
          "y": 0,
          "z": 0
        },
        "roll": 0,
        "x": 0,
        "y": 0,
        "yaw": 0,
```

```
            "z": 0
          }
        }
      }
    },
  "homogeneousCoordinates": {
    "bearing": -3.1415883669408133,
    "distance": 0.4359997212269757,
    "elevation": 0.0011690930720143118,
    "pitch": 0.0011815604520961642,
    "quaternion": {
      "isIdentity": false,
      "w": -0.00000487849775,
      "x": -0.000590780866,
      "y": 0.000133459544,
      "z": 0.9999998
    },
    "roll": 0.00026692497608394846,
    "x": -0.43599942326545715,
    "y": -0.000001868976482910511 5,
    "yaw": -3.141582738898105,
    "z": -0.0005097241373732686
          },
          "pitch": 0.0011815604520961642,
          "roll": 0.00026692497608394846,
          "x": -0.43599942326545715,
          "y": -0.000001868976482910511 5,
          "yaw": -3.141582738898105,
          "z": -0.0005097241373732686
        },
        "unitOfMeasure": "None"
    },
    "statistics": {
      "durationContinuouslyPerceived": "00:00:00",
      "firstPerceived": "2019-09-09T19:14:48.53128Z",
      "mostRecentlyPerceived": "2019-09-09T19:15:10.7358941Z"
    },
    "timeStamp": "2019-09-09T19:15:10.7358941Z"
  },
  {
    "class": null,
    "id": 5,
    "individual": null,
    "isSensible": true,
    "location": {
      "bearing": 2.447178387656334,
    "bearingThreshold": {
      "lowerBound": 0,
      "upperBound": 0
    },
    "distance": 0.13446667538055293,
    "distanceThreshold": {
      "lowerBound": 0,
      "upperBound": 0
    },
    "elevation": -0.511962016172321,
    "elevationThreshold": {
      "lowerBound": 0,
      "upperBound": 0
    },
    "pose": {
      "bearing": 2.447178387656334,
      "created": "2019-09-09T19:15:10.7842637Z",
      "distance": 0.13446667538055293,
      "elevation": -0.511962016172321,
      "frameId": "WorldOrigin",
      "framesProvider": {
        "rootFrame": {
```

```
"created": "2019-09-09T18:30:21.4398142Z",
"id": "RobotBaseCenter",
"isStatic": true,
"linkFromParent": {
  "isStatic": true,
  "parentFrameId": "",
  "transformFromParent": {
    "bearing": 0,
    "distance": 0,
    "elevation": 0,
    "pitch": 0,
    "quaternion": {
      "isIdentity": true,
      "w": 1,
      "x": 0,
      "y": 0,
      "z": 0
    },
    "roll": 0,
    "x": 0,
    "y": 0,
    "yaw": 0,
    "z": 0
  },
  "transformToParent": {
    "bearing": 3.141592653589793,
    "distance": 0,
    "elevation": 0,
    "pitch": 0,
    "quaternion": {
      "isIdentity": true,
      "w": 1,
      "x": 0,
      "y": 0,
      "z": 0
    },
    "roll": 0,
    "x": 0,
    "y": 0,
    "yaw": 0,
    "z": 0
  }
 }
},
"homogeneousCoordinates": {
  "bearing": 2.447178387656334,
  "distance": 0.13446667538055293,
  "elevation": -0.511962016172321,
  "pitch": -1.5696512460708618,
  "quaternion": {
    "isIdentity": false,
    "w": 0.00008819939,
    "x": 0.7066889,
    "y": 0.00009651453,
    "z": 0.7075245
  },
  "roll": 0.21757214098211117,
  "x": -0.09007984399795532,
  "y": 0.07501709461212158,
  "yaw": 2.924042367538678,
  "z": 0.06587369740009308
},
"pitch": -1.5696512460708618,
"roll": 0.21757214098211117,
"x": -0.09007984399795532,
"y": 0.07501709461212158,
"yaw": 2.924042367538678,
```

```
            "z": 0.06587369740009308
          },
          "unitOfMeasure": "None"
        },
        "statistics": {
          "durationContinuouslyPerceived": "00:00:00",
          "firstPerceived": "2019-09-09T19:15:08.9239249Z",
          "mostRecentlyPerceived": "2019-09-09T19:15:10.7842637Z"
        },
        "timeStamp": "2019-09-09T19:15:10.7842637Z"
      }
    ]
  }
}
```


JSON

`WorldState` WebSocket messages are sent even if the data has not changed, as the data is sent via timed updates, instead of being triggered by events. The `WorldState` WebSocket can send data as frequently as every 100ms, though it is set by default to 250ms. To avoid having to handle excess data, you can change the message frequency for the WebSocket with the `DebounceMs` field, as shown in the `lightSocket.js` JavaScript helper.